3 Numerical Solution of differential equations using Simulink[1]

3.0 Introduction

In principle, the solution of a differential equation can be obtained either analytically or numerically. An analytical solution is a closed form solution that exactly satisfies the ODE and the initial condition. A numerical solution, as the name implies, is a set of numerical values of the dependent variable of an ODE, starting at the initial condition. For relatively simply problems, it is possible to calculate the analytical solution. For more complex problems where we have nonlinearities present in the ODEs and the number of ODEs in our mathematical model is large (100's of ODEs will not be unusual) analytical solution becomes impossible. Therefore, we obtain solutions to our ODE's using numerical integration techniques.

3.1 Numerical integration

Consider the following ordinary differential equation, ODE,

$$\frac{dy}{dt} = f(y, u)$$

This describes how the value of the dependent variable, y, changes with respect to time, t. To solve the ODE, we require an initial value, $y(0) = y_0$, i.e. the initial condition, as well as the values of any independent variables $u(t),\ t > 0$. By solution of the ODE, we wish to determine,

$$y(t) = y_0 + \int_0^t f\big(y(t), u(t)\big)dt$$

When we use a numerical integration technique, the calculation is performed over numerous time steps from the start time, at $t = 0$ to the end, $t = t_f$.

All numerical integration techniques are applicable to systems where there is a single, or multiple ODEs. In Chemical Engineering we normally encounter systems where there is more than one differential equation.

3.2 Numerical solution of a set of ODEs

Suppose, we have a mathematical model that consists of a number of nonlinear differential equations, where each of the ODEs is accompanied by a set of initial conditions, i.e. values of the system states at time, $t = 0$,

$$\frac{dy_1}{dt} = f_1(y_1, \dots, y_n)$$
$$\vdots$$
$$\frac{dy_n}{dt} = f_n(y_1, \dots, y)$$

There are many methods that may be used to numerically approximate the analytical solution to a set of differential equations. The simplest, is known as Euler's method. This algorithm starts with the initial conditions and repeatedly increases time by some small positive value evaluating the corresponding outputs based upon a finite difference approximation of the derivatives. The finite difference approximation may be written as,

$$\frac{dy_i}{dt} = \frac{y_i(t_0 + h) - y_i(t_0)}{h} \qquad 3.1$$

[1] Mark J. Willis (mark.willis@ncl.ac.uk)

In equation 3.1, h, is a small time increment, which is usually referred to as the step-size (of the numerical integration routine). The use of this finite difference approximation allows the set of ODEs to be re-written as,

$$y_1(t_0 + h) = y_1(t_0) + hf_1(y_1(t_0), \dots, y_n(t_0))$$
$$\vdots$$
$$y_n(t_0 + h) = y_n(t_0) + hf_n(y_1(t_0), \dots, y_n(t_0))$$

If we specify a step size h, given the initial value of the dependent variables, $y_1(0), \dots, y_n(0)$, we can estimate their values at a time step h in the future. Recursive, or repeated, application of this procedure allows the simultaneous solution of the differential equations, i.e. prediction of the value of the solution of the differential equations at any future time.

For example, at the next time increment we have,

$$y_1(t_0 + 2h) = y_1(t_0 + h) + hf_1(y_1(t_0 + h), \dots, y_n(t_0 + h))$$
$$\vdots$$
$$y_n(t_0 + 2h) = y_n(t_0 + h) + hf_n(y_1(t_0 + h), \dots, y_n(t_0 + h))$$

In this equation, we use the previously obtained estimates of the process outputs to calculate the estimate at time, $t_0 + 2h$. The iterations may be continued to cover the entire time period of interest.

3.3 Example one: Solution of an ODE

Consider the following ODE,

$$\frac{dy}{dt} = 0.2(u - y) \qquad 3.2$$

$$u = \begin{cases} 0.9, t \leq 0 \\ 1.0, t > 0 \end{cases}$$

For $t \leq 0$ we will assume that the system is at steady-state. Therefore, $0.2(u_0 - y_0) = 0$ and the initial condition is, $y_0 = 0.9$.

3.3.1 Analytical solution

To obtain the analytical solution we use Laplace transforms. To ensure zero initial conditions, we define deviation variables from the initial steady-state, i.e.,

$$\bar{y} = y - y_0 = y - 0.9$$

$$\bar{u} = u - u_0 = u - 0.9$$

We then re-write the differential equation in deviation variable form,

$$\frac{d\bar{y}}{dt} = \frac{d(y - y_0)}{dt} = 0.2(u - y) - 0.2(u_0 - y_0) = 0.2(\bar{u} - \bar{y}) \qquad 3.4$$

Taking Laplace transforms, we obtain the transfer function,

$$\frac{\bar{y}(s)}{\bar{u}(s)} = \frac{0.2}{s + 0.2} = \frac{1}{5s + 1} \qquad 3.5$$

Taking inverse Laplace transforms we obtain the analytical solution,

$$\bar{y}(t) = \bar{u}(t)(1 - e^{-0.2t}) \qquad 3.6$$

Given the defined deviation variables and that, $u = 1, t > 0$, we can also write equation 3.6 as,

$$y(t) = 0.9 + 0.1(1 - e^{-0.2t}) \qquad 3.7$$

3.3.2 Using a numerical integration technique

Let us compare the analytical solution with Euler integration. We use a step-size of, $h = 0.2$ seconds. At time, $t = 0$, we have, $y(0) = 0.9$. With $u = 1$ (constant input), we have for $t \geq 0$, $f(u, y) = 0.2(u - y)$. The steps of the numerical integration algorithm are as follows.

At time, $t = t_0 + h$ we make the approximation,

$$y(t_0 + h) = y(t_0) + hf(u(t_0), y(t_0))$$

Therefore,

$$y(t_0 + h) = 0.9 + 0.2\big(0.2(1 - 0.9)\big) = 0.904$$

At time, $t = t_0 + 2h$ we have,

$$y(t_0 + 2h) = y(t_0 + h) + hf(u(t_0 + h), y(t_0 + h))$$

$$y(t_0 + 2h) = 0.904 + 0.2\big(0.2(1 - 0.904)\big) = 0.9078$$

If the iterations are continued we obtain the results shown in Table 3.1, where we compare the values obtained with the exact (analytical) solution values as well as a MATLAB's built in, numerical integrator, *ode45*. We observe slight in-accuracies in our numerical solution.

Euler's method is probably the most basic of all numerical integration methods. It generally, introduces these errors in the calculation of the values of the process output(s) when compared to the exact (analytical) solution. The errors are related to the step size used, i.e. the value of h. If we use a large step size, we would expect larger errors when compared to using a much smaller step size. However, using a small step size would mean that the number of calculations would be increased.

t	y	u	$f(u,y)$	y^*	*ode45*
0	0.9	1	0.02	0.9	0.9
0.2	0.904	1	0.0192	0.9039	0.9039
0.4	0.9078	1	0.0184	0.9077	0.9077
0.6	0.9115	1	0.0177	0.9113	0.9113
0.8	0.9151	1	0.0170	0.9148	0.9148
1.0	0.9185	1	⋮	0.9181	0.9181
⋮	⋮	⋮	⋮	⋮	⋮
29.8	0.9998	1	0.00	0.9997	1.0
30.0	0.9998	1	0.00	0.9998	1.0

Table 3.1. Iteration of Euler's method from time, $t = 0$ to the final simulation time of $t = 30$. The values of y obtained at each time step can be compared to the exact solution, y^*, and we observe there are small errors. More advanced numerical integrators use variable step sizes to control the accuracy of the integration. The final column shows the results obtained using MATLAB's inbuilt (default) integration routine, *ode45*.

3.4 Advanced numerical integration techniques

There are many alternatives to Euler's method, including the Runge- Kutta family of algorithms, variable step size methods and so on. These algorithms generally give more accurate numerical solutions for the differential equations. Advanced integration routines built into MATLAB use variable step size integrators to efficiently and robustly solve the set of ODEs. The recommended numerical integration routine to use within MATAB is *ode45*. *ode45* performs well with most ODE problems. There are, however, a number of alternatives that can be used that can improve solution times particularly when a set of ODEs are stiff.

3.5 Simulink

Simulink (short for Simulation-Link) is a graphics based programming environment for simulating process systems, primarily although not exclusively, dynamic systems. Simulink is a toolbox within MATLAB. It contains a set of block libraries that may be customised and combined in order to simulate system models. Simulink is designed to make process model development intuitive. It offers a means to solve equations numerically using a graphical user interface rather than developing complex code.

Figure 3.1 shows some useful (and commonly used) blocks. The blocks that are shown include a constant block; a mathematical function (the exponential of an input), product, and sum. Simulink allows us to plot data with relative ease using the scope block; we can also display numerical values using a display block and send information out of Simulink to the

MATLAB workspace using the to workspace block.

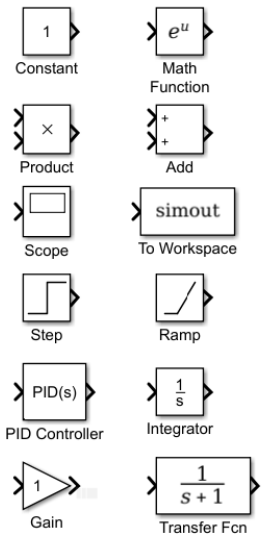


Figure 3.1. Commonly used Simulink blocks. The blocks have input and output ports, our aim is to connect blocks in order to develop a model that can be simulated (solved by numerical integration).

Simulink allows the representation of transfer functions, PI(D) controllers, logical operators etc. The existence of these blocks allows and simplifies the construction of process simulations. To develop a simulation model, our aim is to connect the blocks together in-order to pictorially describe the mathematical equations that we wish to solve numerically.

3.5 Example

We consider the ODE,

$$\frac{dy}{dt} = 0.2(u - y) \qquad 3.3$$

$$u = \begin{cases} 0.9, t \leq 0 \\ 1.0, t > 0 \\ 0.9, t \geq 25 \\ 1.0, t \geq 50 \end{cases}$$

To obtain a numerical solution of an ODE using Simulink, we pictorially represent the equation as shown in Figure 3.2

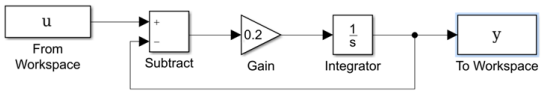


Figure 3.2 Simulink model of the ODE

First we construct the right-hand side of the ODE. We use three blocks; a *from workspace* block that imports the value of u, at the relevant time from the MATLAB workspace; a *subtract* block which subtracts the value of y from u; a *gain* block that multiplies the input signal, $(u - y)$, by a value of 0.2 to give the output signal, $0.2(u - y)$, i.e. the right-hand side of the ODE. To obtain the value of the process output, y, we must integrate the right hand side of the ODE. To do this, we use an *integrator* block (the term, $1/s$, is the Laplace domain representation of an integrator). In order to pass the results of the simulation back to the MATLAB workspace, the final block in figure 3.2 is a *to workspace* block.

we run the simulation from MATLAB, using a script file. Particular points to note regarding the script file are, (1) a variable that is defined in the base workspace, i.e. using a script file is automatically recognised in the Simulink simulation. There is no need to do anything special with them such as declare them as global, (2) the variable y_initial is the initial condition for the *integrator* block. We double click on the *integrator* block and enter the text y_initial, (3) If we use a *from workspace* block then we must define the syntax within the MATLAB workspace. At each discontinuity, a duplicate entry is entered in the time vector and the input vector, which forces the variable-step solver to take a time step at this exact time (using a method known as zero crossings detection which is explained below). The simulation results are shown in Figure 3.3.

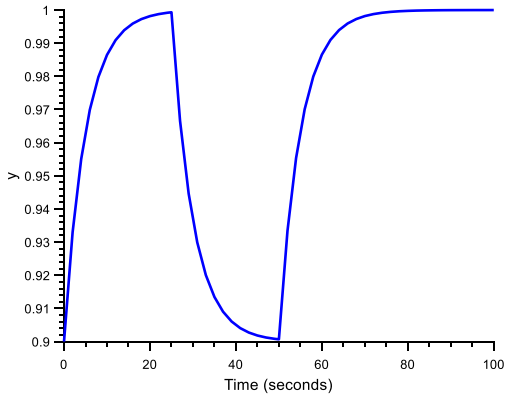


Figure 3.3 Numerical integration (simulation) of the ODE, $\frac{dy}{dt} = 0.2(u - y)$ using Simulink and the MATLAB ode45 routine.

3.6 Zero-crossings detection

We must consider what happens when we introduce discontinuities into a system model, e.g. things such as step-changes in set-point, feed-profile changes in fed-batch reactors or requiring that a simulation stops at a particular time. To model our system accurately we must be aware of when one of these changes is to be made so that the integrator detects and accounts for these changes. Simulink uses a method known as zero-crossing detection to accurately locate a discontinuity and maintain the integrity of the model simulation. It does this by bracketing the zero-crossing event and uses increasingly smaller time steps to pinpoint when the zero crossing has occurred.